

DeepProbe-VLM

Native-looped video SFT: an executable baseline

PRE-EXPERIMENT RELEASE / NO MEASURED GPU RESULTS

Purpose. Establish a faithful, reproducible looped vision-language baseline before testing selective visual-state updates and CUDA co-design. This note replaces neither the earlier frozen-model draft nor a finished results paper. It aligns the new figures with the current implementation and separates working software from research hypotheses.

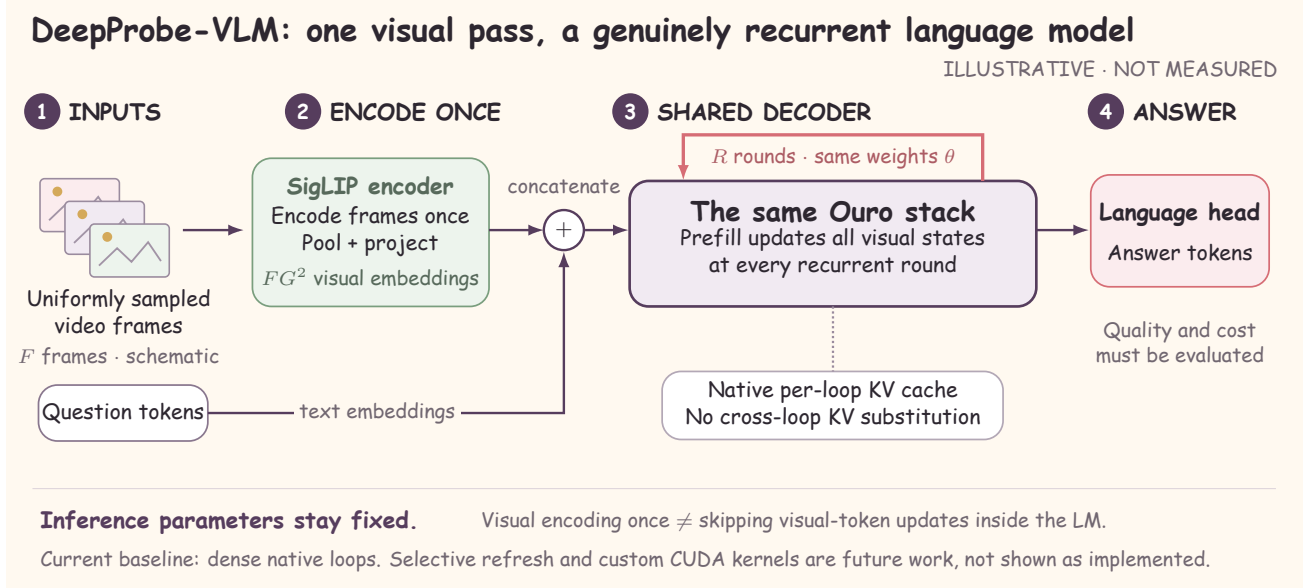


Figure 1: One visual encoding pass, followed by a native recurrent language stack. The question bypasses the vision encoder. During prefill, visual states still evolve at every recurrent round. Diagrams are schematic, not measured evidence.

1. What is implemented

The reference backbone is Ouro-1.4B with its native shared-weight recurrence. SigLIP encodes uniformly sampled frames; a learned spatial pooling/projector produces a bounded visual prefix. An ordinary SmolLM2-1.7B language backbone is also supported as an external comparison. All models are resolved at immutable repository commits.

Training has two stages: projector alignment with frozen backbones, then answer-supervised SFT through the full recurrent language model. The default SFT recipe also unfreezes the vision encoder. The unused adaptive-exit gate stays frozen. This is fixed-depth full-backpropagation SFT, *not* online test-time training or LoRA.

At inference, weights remain fixed. Native caches are distinct across recurrence depths and reused only across generated tokens at their corresponding depth. There is no shallow-to-deep KV substitution, trained halting policy, or custom CUDA/PTX kernel in this release.

Release boundary. Small CPU models test data handling, gradients, checkpoint round trips and cache consistency. No H20/B300 training run, quality improvement, GPU speedup or memory reduction is claimed here.

2. Mathematical contract

For frame features $x_{f,p} = E_\psi(V_f)_p$, the connector uses a positive learned gate and pools within adaptive spatial bins $\mathcal{B}_g$:

$$w_{f,p} = \max\{\sigma(a^\top x_{f,p} + b), \epsilon\}, \quad (1)$$

$$\bar{x}_{f,g} = \frac{\sum_{p \in \mathcal{B}_g} w_{f,p} x_{f,p}}{\sum_{p \in \mathcal{B}_g} w_{f,p}}, \quad v_{f,g} = \text{MLP}_\phi(\bar{x}_{f,g}). \quad (2)$$

With F frames and a $G \times G$ pooling grid, the prefix has $N_v = FG^2$ visual tokens. The starting recipe uses $F = 8$, $G = 4$, hence 128 visual tokens. This is a low-cost temporal-coverage baseline, not evidence that eight frames preserve all long-video events.

Let $H^{(0)}$ concatenate the visual prefix and prompt embeddings. Schematically, the native backbone performs recurrent updates with shared parameters,

$$H^{(r)} = \mathcal{F}_\theta^{\text{native}}(H^{(r-1)}), \quad r = 1, \dots, R. \quad (3)$$

This notation summarizes recurrence; the implementation delegates normalization, residual structure and cache indexing to the pinned Ouro code. It does not emulate loops by repeatedly calling a standard language model. Sharing θ does not imply identical $H^{(r)}$, keys, or values across rounds. Under causal masking, the visual prefix cannot read later question tokens: current visual updates are not query-adaptive.

The supervised objective is final-round, answer-only causal cross-entropy:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} \log p_{\theta, \phi, \psi}(y_t \mid V, q, y_{<t}; R). \quad (4)$$

Prompt, visual-prefix and padding positions are ignored. The wrapper requests final-round logits and computes the shifted loss itself; it does not silently inherit a different weighted-exit objective. Gradients flow through all rounds without inter-round detachment.

3. What would constitute a research contribution?

The baseline components above are not claimed as novel. A future contribution must demonstrate a new mechanism and a controlled quality–cost trade-off beyond this dense reference:

- **Selective visual-state refresh:** train an internal update policy, preserving a valid dense fallback and cache semantics. Merely selecting frames before the model is a different problem.
- **Budget-aware learning:** study an explicit quality–compute objective, then check whether its compute proxy predicts actual latency. No unproved mathematical guarantee is claimed.
- **Kernel co-design:** implement compaction, scheduling or fusion only for a mechanism shown useful in profiling; validate both numerical behavior and end-to-end quality.

These branches are hypotheses. Sparse masks alone do not show skipped GPU work, and weight sharing alone does not justify cross-round cache reuse.

4. Training and evaluation protocol

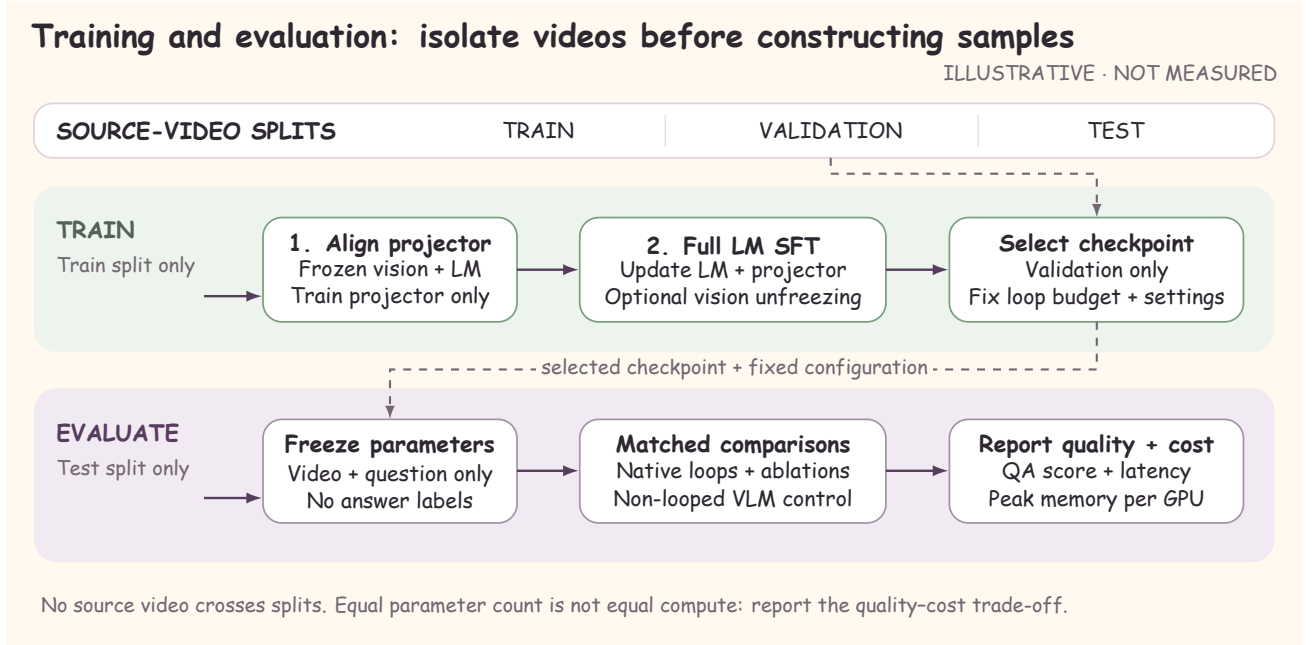


Figure 2: Separate offline optimization and held-out evaluation. Checkpoint provenance carries source-video identities through alignment and SFT. Test questions do not become training examples or checkpoint-selection criteria.

Resource	Intended role	Required qualification
LLaVA-Video-178K	Training and source-disjoint development split	Start with an explicit subset; preserve upstream media/source identities.
Video-MME	Held-out multiple-choice evaluation	Report no-subtitle setting, sample count, and short/medium/long breakdown.
MVBench	Held-out task-level evaluation	Respect unavailable/licensed media; label incomplete task coverage as a subset.

The manifest records a sample ID, original video ID, relative media path, split, question and target. Clips from the same original video must remain in the same split. The validator detects duplicate normalized IDs and media-source conflicts; it is not a universal content-duplicate detector. Ambiguous upstream IDs require an explicit mapping rather than invented certainty.

The downloader defaults to planning only. It pins repository revisions, estimates transfer size, enforces an explicit size cap and requires opt-in before downloading. Dataset and model licenses remain the user’s responsibility; media and weights are not republished in the code repository.

Metrics. Multiple-choice accuracy is separate from normalized exact match for open answers. The latter is not an official semantic judge. For confidence intervals, resample source videos rather than question rows. Paired comparisons must contain identical samples, targets, splits and source identities.

Selection. Select hyperparameters/checkpoints on the development split. Save ancestor training and tuning provenance inside each checkpoint. Final evaluation must reject source overlap with either training or tuning. Missing-media counts and failed examples must be reported, not silently dropped.

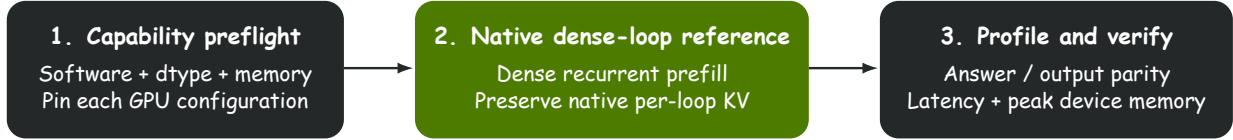
5. H20/B300 execution and measurement

H20 / B300 runtime: establish a faithful baseline before kernel work

ILLUSTRATIVE · NOT MEASURED

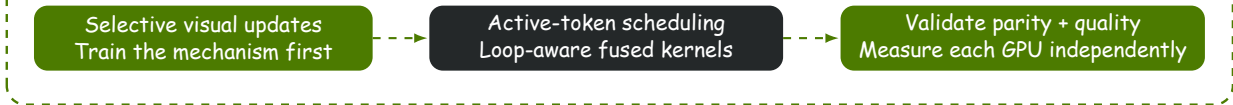
CURRENT EXPERIMENT PATH

Logical execution flow · not a GPU topology or timeline



FUTURE WORK · NOT IMPLEMENTED

Proceed only if profiling identifies a material bottleneck



No H20/B300 speedup is established. A sparse mask alone does not prove skipped computation or lower latency.

Figure 3: White-background, filled green/charcoal runtime diagram. The top row is the current logical execution path; the dashed lower row is future work. This is not a physical GPU topology or measured execution timeline.

Gate sequence. Check the actual device, driver and CUDA/PyTorch build. Test a BF16 attention forward and backward pass, then verify an eight-device collective. Run a 20-step model pilot on one GPU and then on eight. Inspect finite loss, all expected gradients, peak memory, checkpoint resume and cached/uncached output parity before committing a large budget.

The included launcher uses ordinary replicated DDP, FP32 trainable weights/Adam state and BF16 autocast. It does not claim FSDP/ZeRO support. Gradient checkpointing reduces activation storage but does not remove recurrent compute. Default full-SFT microbatch is one per device with 16 accumulation steps: global batch 128 on eight GPUs.

Timing boundary. The reference evaluator records preparation, model execution and end-to-end elapsed time, plus peak allocated memory. Model time includes vision encoding, prefill and decode; those components are not separately instrumented yet. Raw timing is cold/mixed and profiler traces add overhead. Neither is a warmed throughput result.

For a publishable systems comparison, use fixed inputs, decoding length, batch, precision and software provenance; add warmup and repeated synchronized measurements. Report median/p95 latency, actual generated tokens, memory and task quality. Measure each GPU family independently. Estimate remaining training time only from observed steady-state step time, adding validation and checkpoint costs.

6. Minimal ablation matrix

Axis	Controlled comparison	Interpretation
Recurrent depth Ordinary model	Ouro $R = 1, 2, 4$, separately adapted SmolLM2, same visual pipeline	Quality and cost of native recurrent depth. External reference; pretraining/size confounds remain.
Vision adaptation	Frozen vs trainable encoder	Whether end-to-end visual adaptation matters.
Visual budget	$F = 4, 8, 16$; $G = 2, 4, 8$	Change one axis at a time; temporal vs spatial budget.
Alignment Cache	With vs without connector alignment Cached vs full recomputation	Include an equal-total-budget comparison. Correctness first; compare same checkpoint/output.
Hardware	H20 vs B300	Matched workload, measured independently.

Use seeds 17, 29 and 43 for final model comparisons. An initial pilot is not a multi-seed result. Report both example/step-matched and compute-matched comparisons where feasible: a larger R consumes more computation even at the same parameter count. Changing recurrence depth only at inference is not equivalent to training at that depth.

Avoid an invalid claim. Ouro and SmolLM2 differ in pretraining, tokenizer, parameter count, architecture and initial instruction behavior. A higher benchmark score against that model would be useful evidence, but not causal proof that looping is the sole reason. Count the vision encoder and connector when reporting total VLM parameters.

7. Reproducibility checklist

- Model and dataset commits, manifest hashes, source-disjoint split policy.
- Training stage, all optimizer/schedule settings, seed, effective global batch and sample budget.
- Checkpoints with optimizer/RNG state for exact resume; weights-only initialization for stage changes.
- GPU/driver/CUDA/library versions, actual parameter counts and validation logs.
- Per-example predictions, strict metric definitions, coverage and source-cluster confidence intervals.
- No fabricated result tables or forecast improvements presented as observations.

Official implementation and data references

These are primary resource links for the current baseline, not a complete related-work bibliography. Model commits and data revisions are specified in repository configuration files.

- [ByteDance: Ouro-1.4B model and native recurrent implementation.](#)
- [Google: SigLIP SO400M patch14-384.](#)
- [Hugging Face: SmolLM2-1.7B-Instruct.](#)
- [LLaVA-Video-178K dataset.](#)
- [Video-MME official project; MVBench / VideoChat2 official project.](#)